

23B0983-23B0963-23B0920

1 Phase 1

`compute_rolling_hashes`

Description: Computes rolling hashes for fixed-length substrings in a submission for efficient comparison and stores starting indices in a map.

Input: Token vector, substring length.

Output: Hash map of hash values to starting indices.

Complexity: $O(n)$ time and space.

`find_matches`

Description: Identifies exact matches of minimum length(5) between two submissions and considers it only if ≥ 10 length.

Input: Two token vectors, minimum length, and match index storage.

Output: Total length of exact matches.

Method: Rolling hash comparison with overlap checks and visited map.

Complexity: $O(n * m)$ time where n is length of file1 and m is length of file 2 and space is $O(n+m)$.

`find_maximum_approximate_matches`

Description: Extends exact matches back first and later front until gap fraction exceeds threshold (0.80) to get approximate matches

Input: Two token vectors, exact match indices, gap fraction.

Output: Longest approximate match length and starting indices.

Method: Expand matches iteratively, allowing gaps.

Complexity: $O(k \cdot n \cdot g)$ time, $O(1)$ space, where k is the number of initial matches, g is the maximum allowable gap and n is the smallest submission size.

Overall Flow

1. Compute rolling hashes for exact matches (`compute_rolling_hashes`).
2. Identify exact matches (`find_matches`).
3. Expand to approximate matches (`find_maximum_approximate_matches`).
4. Return the results.

2 Phase 2

Basic idea: All 15-length grams are hashed and stored in a hash_map, each hash value maps to all possible locations and tokens comparison is achieved through hash value comparison

Long Pattern Match Detection

- **Algorithm:** For each hash of the file, its matching is checked through search in the hash_map and if a search is found, manually check if 75 tokens from that position are matching.
- **Key Functions:**
 1. `check_for_plag_3()`: Main driver for long pattern detection
 2. `search_for_hash_3()`: Verifies 75+ consecutive token matches
- **Complexity:** $O(75*n)$ time, $O(3*n)$ space where n = tokens/file

Patchwork Plagiarism Detection

- **Algorithm:** For each hash in the file, it is checked if it matches with anyone location, another map is used to capture already mapped locations.
- **Key Functions:**
 1. `check_for_plag_1_2()`: Main driver for short pattern detection and patchwork detection
 2. `search_for_hash2()`: checks if a given hash has a match
- **Complexity:** $O(3*n)$ time, $O(2*n)$ space where m = files, n = tokens/file

Individual File Short Pattern Detection

- **Algorithm:** for each hash in the file, it is mapped to all possible valid submissions(with almost one map for each submission) and a map is maintained which maps each submission to the number of matches to has with actual file
- **Key Functions:**
 1. `check_for_plag_1_2()`: Main driver for short pattern detection and patchwork detection
 2. `search_for_hash1()`: Checks and maps each hash to all valid locations
- **Complexity:** $O(n+m)$ time, $O(2*m+2*n)$ space where m = files, n = tokens/file

Threading and Concurrency

1. Non-blocking achieved through producer-consumer pattern:
 - `add_submission()`: Producer thread queues submissions to `tokenize_queue`
 - `process_tokenization()`: Consumer thread processes tokens and hashes
 - `process_submission()`: Final consumer checks for plagiarism
2. Protected by mutex locks and condition variables for thread safety

Time Window Management

- Timestamps stored in `map_timestamp` with shared mutex
- Files within 1-second window identified during hash matching
- Window checks performed in `search_for_hash1/2/3` functions

System Architecture

1. Data Structures:
 - `map_tokens`: Stores tokenized submissions
 - `hash_map`: Maps hash values to submission locations
 - `map_timestamp`: Tracks submission timestamps
 - Thread-safe queues for processing pipeline
2. Flow:
 - Constructor initializes data structures and worker threads
 - New submissions enter tokenization queue
 - Tokenizer processes and computes hashes
 - Plagiarism checker performs all detection methods
 - Destructor ensures queue completion and cleanup

Protected by comprehensive mutex system and shared locks for concurrent access safety.